Chapter 1 Introduction

1.1 Introduction

Life+Link System is a web-based healthcare and organ donation management platform developed using Python, Streamlit, and SQLite database. The main objective of the system is to simplify organ donation management, hospital communication, emergency organ requests, and inventory management through a centralized digital platform.

The system provides separate modules for users, hospitals, organ banks, and administrators. Users can register as organ donors and manage their profiles. Hospitals can send organ requests, manage emergency cases, and communicate with organ banks. Organ banks can maintain donor records and organ inventory, while administrators can monitor the entire system using analytics dashboards.

Traditional organ donation systems mainly depend on manual paperwork and offline communication, which creates delays during emergency situations. The Life+Link System helps in reducing manual work and improving healthcare communication through automation and centralized management.

The project is developed using Streamlit for frontend interface development and SQLite database for storing all healthcare records. The application also includes blood group compatibility matching, certificate generation, sound notifications, and dashboard analytics.

1.2 Motivation

The motivation behind developing the Life+Link System comes from the difficulties faced by hospitals and healthcare organizations during emergency organ request handling. In many healthcare systems, donor records and emergency communication are maintained manually, which increases complexity and delays response time.

Organ donation requires proper communication between hospitals, organ banks, and donors. However, traditional systems often lack centralized management and real-time inventory

tracking. Patients may face delays in finding suitable organ donors because of inefficient record handling and communication systems.

This project aims to provide a digital solution that simplifies organ donation management and improves healthcare communication. The system provides donor matching, centralized inventory management, and emergency request handling through an organized web platform.

1.3 Purpose

The purpose of the Life+Link System is to provide an efficient healthcare management platform for organ donation and emergency request management. The system helps users register as donors while allowing hospitals and organ banks to manage requests and inventory digitally.

The platform provides secure authentication and centralized database management for maintaining donor records, hospital requests, emergency cases, and inventory details. The project also aims to reduce paperwork and improve communication between hospitals and organ banks.

Another purpose of the project is to provide analytics dashboards for administrators and support blood group compatibility matching during emergency situations.

1.4 Objectives

The objectives of the Life+Link System are:

- To develop a centralized healthcare management platform.
- To provide role-based login functionality.
- To manage organ donor registration digitally.
- To handle emergency organ requests efficiently.
- To maintain organ inventory records.
- To provide blood group compatibility matching.
- To generate dashboard analytics for monitoring.
- To reduce manual paperwork and management complexity.

Chapter 2 Literature Survey

2.1 Existing System

Most healthcare organizations currently use manual methods for managing organ donation records and emergency communication. Hospitals maintain donor and patient records separately, which creates difficulties in data management and emergency coordination.

Existing healthcare systems usually lack centralized inventory management and donor matching functionality. Emergency organ requests require quick communication between hospitals and organ banks, but traditional systems often fail to provide fast response management.

2.1.1 Elaborate on Existing System Applications

Many hospitals use spreadsheets, registers, and basic healthcare software for maintaining donor and patient records. Some healthcare organizations use standalone management applications for inventory and patient details.

A few organ donation organizations provide donor registration through websites, but these systems generally do not include advanced features like emergency request handling, donor matching, analytics dashboards, and centralized communication.

Most existing systems focus only on patient management and do not provide complete organ donation and healthcare inventory management solutions.

2.1.2 Limitations or Challenges in Existing System

The existing systems have several limitations:

1. Manual record management increases complexity.
2. Lack of centralized communication between hospitals and banks.
3. Delayed emergency request handling.

4. Difficult inventory management.
5. No proper donor matching system.
6. Limited analytics and monitoring support.
7. Increased chances of data redundancy and errors.

2.2 Problem Statement

Hospitals and healthcare organizations face difficulties in managing organ donor records and emergency organ requests efficiently. Traditional systems depend on manual communication and paperwork, which reduces transparency and delays emergency healthcare services.

Organ banks also face challenges in maintaining donor inventory and matching donors with recipients quickly. Existing systems do not provide centralized donor matching, analytics dashboards, and emergency management support.

The Life+Link System aims to solve these problems by providing a centralized web-based healthcare platform with donor management, inventory tracking, emergency request handling, and analytics features.

2.3 Feasibility Study

Technical Feasibility

The project is technically feasible because it uses Python, Streamlit, and SQLite, which are lightweight and easy-to-use technologies. The required software tools are open-source and can run on standard computer systems.

Economic Feasibility

The project is economically feasible because it uses free and open-source technologies. No expensive hardware or commercial software is required.The system is user-friendly and easy to operate for users, hospitals, organ banks, and administrators. The platform provides centralized healthcare management with secure authentication and dashboard support.

Chapter 3 Project Scope and Requirement Analysis

3.1 Requirement Gathering and Analysis

Requirement gathering and analysis is an important phase in the development of the Life+Link System. The requirements were identified by studying the problems faced in traditional healthcare and organ donation management systems and understanding the need for a centralized digital platform for hospitals, organ banks, and users.

The analysis showed that hospitals require a fast and organized system to manage organ requests, emergency cases, and patient records efficiently. Organ banks also require a platform to maintain donor inventory, monitor organ availability, and communicate with hospitals during emergency situations. Users require a simple system to register as organ donors and manage their profiles digitally.

Based on these observations, the project requirements were finalized. The system should provide secure user authentication, donor registration, hospital request management, organ inventory handling, blood group compatibility matching, analytics dashboards, and emergency request management. The analysis also identified the need for a user-friendly interface, centralized database storage, and efficient communication between hospitals and organ banks.

Overall, the requirement analysis helped in designing a secure, efficient, and user-friendly healthcare and organ donation management system that reduces manual paperwork and improves emergency healthcare management.

3.2 Software Requirements

1. Frontend Framework: Streamlit
   Used for developing the user interface, dashboards, forms, and interactive web pages of the Life+Link System.

2. Backend Language: Python
   Used for implementing backend logic, authentication, database operations, donor matching, and healthcare management functionalities.

3. Database: SQLite3
   Used for storing user records, hospital details, organ donor information, emergency requests, inventory records, and analytics data.

4. IDE/Editor: Visual Studio Code
   Used for writing, editing, debugging, and executing Python code.

5. Operating System: Windows
   The application is designed to run on Windows operating systems.

6. Python Libraries Used:

   • Streamlit
   • sqlite3
   • pandas
   • hashlib
   • os
   • datetime

7. Browser: Google Chrome
   Used for running and testing the Streamlit web application.

3.3 Hardware Requirements

The Life+Link System can be used by users on both mobile phones and computers, while hospitals, organ banks, and administrators mainly use desktop or laptop systems for managing healthcare operations and dashboards. The hardware requirements for smooth execution of the application are as follows:

- **Processor:** Dual Core Processor or Higher
  Required for smooth execution of the application and healthcare management operations.

- **RAM:** Minimum 4 GB
  Required for frontend, backend, and database operations efficiently.

- **Storage:** Minimum 5 GB Free Space
  Used for storing project files, healthcare records, donor information, and inventory data.

- **Mobile Device:** Android Smartphone or iPhone
  Users can access the system through mobile devices for donor registration, request tracking, and profile management.

- **Keyboard and Mouse:**
  Required for data entry, navigation, and managing dashboards on desktop or laptop systems.

- **Monitor/Display Screen:**
  Required for displaying application dashboards, reports, forms, and healthcare management interfaces.

- **Power Supply:**
  Continuous power supply is necessary for uninterrupted system operation.

- **Internet Connection:**
  Required for accessing the application, downloading dependencies, and maintaining communication between users, hospitals, and organ banks.

3.4 Function requirements

a) User Defined Function

1. **User Authentication Function**
   The system should allow users, hospitals, banks, and administrators to register and login securely.

2. **Role Management Function**

   The system should differentiate between User, Hospital, Organ Bank, and Admin roles.

3. **Donor Registration Function**

   Users should be able to register as organ donors and manage donor details digitally.

4. **Hospital Request Management Function**

   Hospitals should be able to create and manage organ requests and emergency cases.

5. **Inventory Management Function**

   Organ banks should be able to maintain organ inventory and availability records.

6. **Blood Group Matching Function**

   The system should match donors and recipients based on blood group compatibility.

7. **Emergency Request Function**

   Hospitals should be able to send emergency organ requests quickly and efficiently.

8. **Analytics Dashboard Function**

   The system should provide analytics dashboards for monitoring healthcare activities and inventory records.

9. **Certificate Generation Function**

   The system should generate digital certificates for registered organ donors.

10. **Session Management Function**

    The system should securely maintain login sessions for all users and administrators.

11. **Notification Function**

    The system should provide sound notifications and alerts for important actions and emergency activities.

12. **Database Management Function**

    The system should securely store and retrieve donor records, inventory data, and request information using SQLite database.

b) System Functions

1. **Session Management Function**

   The system should securely maintain login sessions for all users and administrators.

2. **Database Management Function**

   The system should securely store and retrieve donor records, inventory data, and request information using SQLite database.

3. **Authentication Verification Function**

   The system should validate login credentials and provide secure access control.

4. **Data Processing Function**

   The system should process donor matching, emergency requests, and inventory operations efficiently.

5. **Analytics Processing Function**

   The system should generate reports, charts, and analytics dashboards for monitoring healthcare activities.

3.5 Non-Functional Requirements

1. **User-Friendly Interface**

   The system should provide a simple and easy-to-use interface for all users.

2. **Performance**

   The application should respond quickly during donor registration, request handling, and dashboard operations.

3. **Security**

   User data, donor records, and login credentials should be stored securely.

4. **Reliability**

   The system should perform healthcare management operations without major errors or failures.

5. **Maintainability**

   The project should be easy to maintain, update, and improve in the future.

6. **Scalability**

   The system should support future enhancements such as cloud integration and mobile application support.

7. **Responsiveness**

   The application interface should work properly on different screen sizes and devices.

8. **Availability**

   The system should be available whenever hospitals, users, or administrators access it.

9. **Database Integrity**

   The system should maintain accurate and consistent healthcare records in the database.

Chapter 4 :Project Design and Modeling Details

4.1 Software Requirement Specification (SRS):

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the Life+Link System. It describes the overall behavior , features, modules, and system operations required for the successful implementation of the healthcare and organ donation management platform.

The Life+Link System is designed to provide a centralized web-based solution for users, hospitals, organ banks, and administrators. The system should provide secure authentication, donor registration, emergency request handling, inventory management, donor matching, analytics dashboards, and healthcare communication functionalities.

The frontend of the system is developed using Streamlit, which provides interactive dashboards, forms, and user interfaces for healthcare management operations. The backend is developed using Python programming language, which handles authentication, request processing, donor matching, inventory management, analytics generation, and database communication. SQLite3 database is used for storing healthcare records securely.

The system should support role-based access control for Users, Hospitals, Organ Banks, and Administrators. Users should be able to register as organ donors and manage profiles. Hospitals should be able to create organ requests and manage emergency cases. Organ banks should maintain organ inventory and donor records, while administrators should monitor the complete system through analytics dashboards.

The system should also provide secure database storage, efficient performance, responsive interface design, and proper communication between all modules. The SRS document helps in understanding the project requirements clearly and acts as a guideline during system development and implementation.

4.2 System Architecture

The Life+Link System follows a client-server architecture. The frontend is developed using Streamlit, which provides the graphical user interface for users, hospitals, organ banks, and administrators. Users interact with the system through web pages such as login forms, donor registration forms, dashboards, inventory management pages, and emergency request sections.

The backend is developed using Python programming language. Python handles application logic, donor matching functionality, user authentication, routing, database communication, emergency request processing, and analytics generation. It acts as a bridge between the frontend interface and the database layer.

SQLite3 is used as the database layer for storing user details, donor records, hospital requests, organ inventory information, emergency requests, and analytics data securely.

The architecture ensures proper communication between the frontend, backend, and database. The modular structure improves maintainability, reliability, scalability, and overall system performance.

4.3 System Modeling and Design

The Life+Link System is designed using a modular and centralized healthcare management approach. The system modeling defines how users, hospitals, organ banks, and administrators interact with the application and how data flows between different modules and the database.

The system begins with user authentication, where users select their role and login securely into the application. After successful authentication, users are redirected to their respective dashboards based on their roles.

The User Module allows organ donor registration, profile management, and request tracking functionalities. The Hospital Module handles patient records, emergency organ requests, and donor matching processes. The Organ Bank Module manages organ inventory, donor availability, and emergency request processing. The Admin Module monitors the complete system using analytics dashboards and reports.

The Donor Matching Module processes blood group compatibility and identifies suitable donors for hospitals during emergency situations. The Notification Module generates alerts and sound notifications for important activities such as successful login, request submission, and emergency actions.

The data flow of the system includes user input, request processing, database storage, donor matching, inventory verification, and dashboard visualization. SQLite database stores all healthcare records securely and provides efficient retrieval of donor information, inventory details, and emergency requests.

The system modeling and design improve communication between modules and ensure smooth healthcare management operations through centralized database connectivity and secure data handling.

4.4 Data Structures used

The Life+Link System uses different data structures for storing, managing, and processing healthcare records efficiently. These data structures help in organizing donor information, hospital records, inventory details, emergency requests, and analytics data systematically.

1. Lists

Lists are used for storing collections of donor records, hospital requests, inventory details, and analytics information temporarily during program execution. Lists help in managing multiple records dynamically and allow easy insertion and retrieval of healthcare data.

2. Dictionaries

Dictionaries are used for storing user details, donor information, hospital data, and inventory records in key-value pair format. This structure provides fast data access and improves data management efficiency within the system.

3. SQLite Database Tables

SQLite database tables are used for permanent storage of healthcare records such as users, hospitals, organ requests, inventory details, emergency requests, and administrator data. Database tables help maintain data consistency, security, and structured storage.

4. Pandas DataFrames

Pandas DataFrames are used for analytics dashboard generation, healthcare statistics, and report visualization. DataFrames help in organizing tabular data efficiently and support chart generation for administrators.

5. Strings

Strings are used for handling usernames, passwords, blood groups, organ names, addresses, emergency status messages, and notification messages throughout the application.

6. Integer and Float Variables

Integer and float data types are used for storing quantities, IDs, counts, analytics values, and inventory statistics within the system.

The combination of these data structures improves system performance, data organization, analytics processing, and efficient healthcare management operations.

Chapter 5: Implementation

5.1 Algorithms

1. User Authentication Algorithm

The User Authentication Algorithm accepts login credentials such as username, email, and password from users, hospitals, organ banks, or administrators. The system validates the entered credentials and compares them with database records. If the credentials are correct, the user is redirected to the appropriate dashboard based on the selected role. Otherwise, an error message is displayed for invalid login attempts

2. Donor Registration Algorithm

The Donor Registration Algorithm allows users to register as organ donors by entering personal information such as blood group, contact details, and organ donation preferences. The system validates the entered data and stores donor records securely in the SQLite database

3. Donor Matching Algorithm

The Donor Matching Algorithm accepts recipient blood group and organ request details from hospitals. The system compares donor blood groups with recipient requirements and identifies compatible donors based on blood group matching rules. Matching donor records are then displayed for emergency processing.

4. Emergency Request Algorithm

The Emergency Request Algorithm allows hospitals to create emergency organ requests. The system stores request details in the database and verifies organ availability from the organ bank inventory. The request status is updated dynamically according to donor and inventory availability.

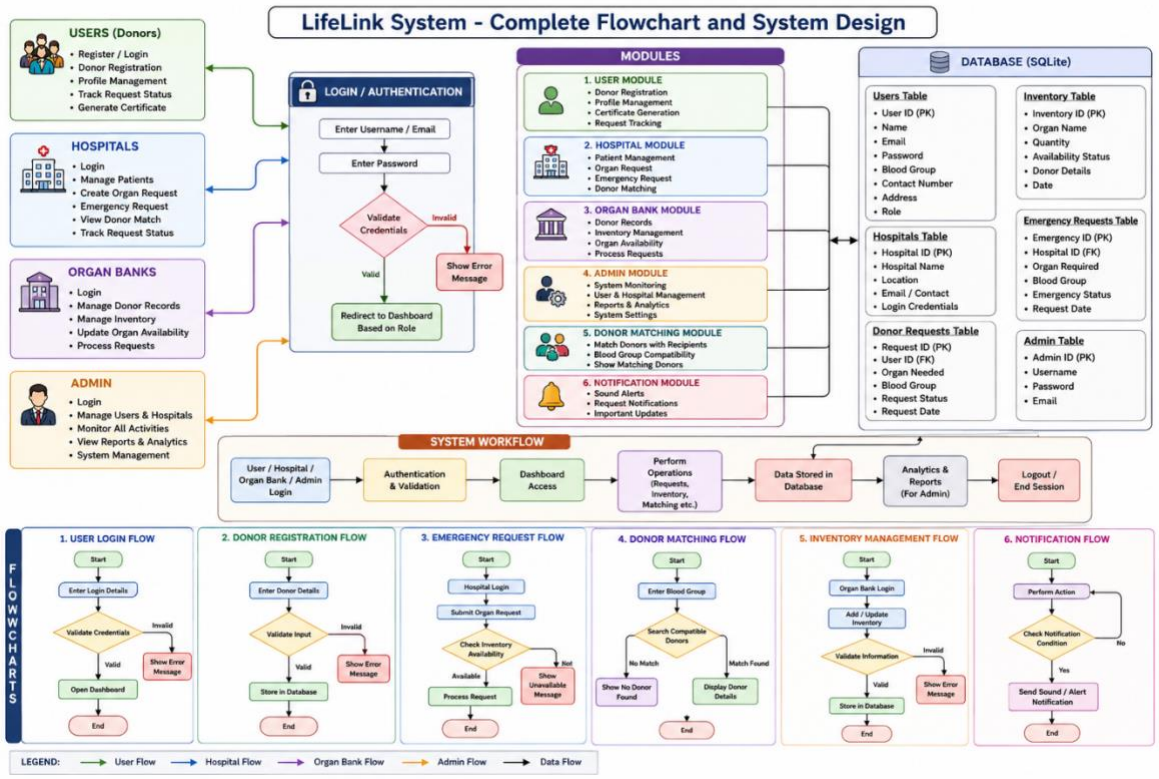
5. Inventory Management Algorithm

The Inventory Management Algorithm allows organ banks to add, update, and manage organ inventory records. The system maintains organ quantity, donor details, and availability status efficiently in the database.

6. Analytics Dashboard Algorithm

The Analytics Dashboard Algorithm collects healthcare records, donor statistics, request details, and inventory data from the database and generates graphical reports and analytics charts for administrators.

5.2 Flowcharts



Chapter 6 :Testing

6.1 Input Validation

The Life+Link System performs input validation to ensure secure and accurate data entry throughout the healthcare management process. During registration, the system verifies whether all required fields are entered properly. Duplicate email or username registration is restricted to prevent multiple accounts with the same credentials.

During login, the system validates usernames, emails, and passwords with database records and displays appropriate error messages for invalid login attempts. In the donor registration module, the system validates blood group details, contact information, and organ donation preferences before storing records in the database.

The emergency request module validates hospital details, organ request information, and blood group data before processing emergency requests. The donor matching module checks blood group compatibility before displaying matching donor records.

The inventory management module validates organ quantity, availability status, and donor details before updating database records. The certificate generation functionality also verifies donor information before generating digital certificates.

Input validation improves system security, reliability, database integrity, and healthcare record accuracy by preventing invalid, incomplete, or incorrect data from being stored in the system.

Chapter 7 : Conclusion

The Life+Link System successfully provides a simple, secure, and efficient web-based healthcare and organ donation management platform. The project helps users register as organ donors, manage donor details, and access healthcare services through a centralized digital system.

The platform also helps hospitals manage patient records, emergency organ requests, and donor matching operations efficiently. Organ banks can maintain donor inventory, process requests, and monitor organ availability through the system. The administrator module provides analytics dashboards and monitoring features for better healthcare management and reporting.

The use of Python, Streamlit, and SQLite3 provides secure database management, smooth system functionality, and interactive user interface support. The project reduces manual paperwork and improves communication between users, hospitals, and organ banks through digital automation.

The donor matching functionality based on blood group compatibility improves emergency healthcare response and simplifies organ request processing. The organized database structure and responsive interface improve overall system performance, usability, and reliability.

Future improvements may include cloud database integration, mobile application support, SMS and email notifications, real-time hospital tracking, and advanced AI-based donor prediction systems. Overall, the Life+Link System fulfills its objective of providing a user-friendly, reliable, and centralized healthcare and organ donation management platform.

References

a) Journal/Conference Papers

1. Research Papers on Healthcare Management Systems
2. Organ Donation and Transplantation Management Research Articles
3. Healthcare Analytics and Emergency Response System Journals
4. Research Papers on Web-Based Healthcare Applications
5. Digital Healthcare Communication System Research Papers

b) Book References

1. *Python Programming* by Reema Thareja
2. *Database System Concepts* by Abraham Silberschatz
3. *Software Engineering* by Roger Pressman
4. *Fundamentals of Database Systems* by Elmasri and Navathe
5. *Web Technologies* by Uttam K. Ro

c) Web References

1. Python Official Documentation:
   https://docs.python.org/3/
   Used for Python programming concepts, Streamlit integration, database handling, and backend development.

2. **Streamlit Official Documentation**:
   sqlite3 — DB-API 2.0 interface for SQLite databases — Python 3.14.5 documentation
   Reference for Streamlit components, dashboard creation, forms, session handling, and web application development.

3. **SQLite3 Documentation:**
   https://docs.python.org/3/library/sqlite3.html
   Used for database creation, SQL queries, and database operations.

4. **SQLite Official Website**:
   https://www.sqlite.org/
   Reference for relational database concepts and SQLite database management.

5. **W3Schools:**
   https://www.w3schools.com/
   Used for frontend concepts, Python basics, forms, and interface design references.

6. **Visual Studio Code Documentation**:
   https://code.visualstudio.com/docs
   Used for project development environment and code management.